SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1019	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Yamuna K	**Reg. No.:**	192511013
Date:	20/10/2026	**Duration:**	60 minutes

Code of Conduct

I **YAMUNA K/192511013** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *Yamuna K*

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ANNEXURE A – ARCHITECTURE DESIGN ASSIGNMENT

Project Title: Smart Parking Management System
Domain: Transport and Parking Management

1. Analyze System Requirements

1.1 Introduction

The Smart Parking Management System is a software solution designed to simplify parking operations in malls, colleges, and campuses. Drivers often waste time searching for available parking spaces, causing traffic congestion, fuel wastage, and inconvenience.
The proposed system displays real-time parking slot availability, supports advance booking, automates vehicle entry and exit billing, and provides occupancy analytics for parking operators.

1.2 System Objectives

1. Display real-time availability of parking slots.
2. Allow drivers to book parking slots in advance.
3. Automate vehicle entry, exit, and parking fee calculation.
4. Provide secure digital payment and booking confirmation.
5. Generate occupancy and revenue reports for parking operators.

1.3 Functional Requirements

ID	Functional Requirement	Description
FR1	User Management	Register users and authenticate login.
FR2	Slot Availability	Display available, occupied, and reserved slots.
FR3	Advance Booking	Allow drivers to reserve parking slots.
FR4	Entry and Exit	Record vehicle entry and exit using sensors, RFID, or cameras.
FR5	Automated Billing	Calculate parking fees based on parking duration and configured rates.
FR6	Digital Payment	Process payments through an external payment gateway.
FR7	Slot Management	Allow operators to manage slots and parking rates.
FR8	Analytics	Generate occupancy, usage, and revenue reports.
FR9	Notifications	Send booking confirmations and payment updates.

1.4 Non-Functional Requirements

Requirement	Description
Scalability	Support increasing numbers of users, slots, and parking locations.
Security	Protect user information and payment-related data.
Performance	Display slot availability quickly and process bookings efficiently.
Reliability	Maintain accurate booking, entry/exit, and billing records.
Maintainability	Allow components to be updated independently where possible.
Availability	Keep the service accessible during normal parking operations.

1.5 Quality Attributes Influencing Architecture

- Scalability: The architecture must support multiple parking locations and growing user demand.
- Security: Authentication, authorization, encrypted communication, and secure payment integration are required.
- Performance: Slot updates and booking operations should have low response times.
- Reliability: The system must prevent double booking and inconsistent billing.
- Maintainability: Separate components should simplify testing, debugging, and future modifications.
- Availability: Failures in individual components should not unnecessarily stop the entire system.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Hybrid Architecture

The proposed system uses a Hybrid Architecture combining Layered Architecture, Client–Server Architecture, and Event-Driven Architecture.

- Layered Architecture: Separates the user interface, business logic, and data access.
- Client–Server Architecture: Allows web and mobile clients to communicate with a centralized backend through APIs.
- Event-Driven Architecture: Supports asynchronous processing of sensor updates, booking notifications, and analytics events.

2.2 Justification for Selecting the Architecture

1. Separation of concerns: The presentation, business logic, and database operations are separated into distinct layers.
2. Real-time updates: Sensor events can update parking slot availability without requiring every component to communicate directly with every other component.
3. Scalability: The application and event-processing components can be scaled according to demand.
4. Integration: External services such as payment gateways, RFID readers, and parking sensors can communicate through defined interfaces.
5. Maintainability: Changes to the payment service or analytics module can be made with limited impact on other components.

This architecture is more suitable than a simple standalone application because the system needs real-time updates, external integrations, and support for multiple parking facilities.

3. Software Architecture Diagram

The following diagram illustrates the main components, communication paths, database, and external systems.

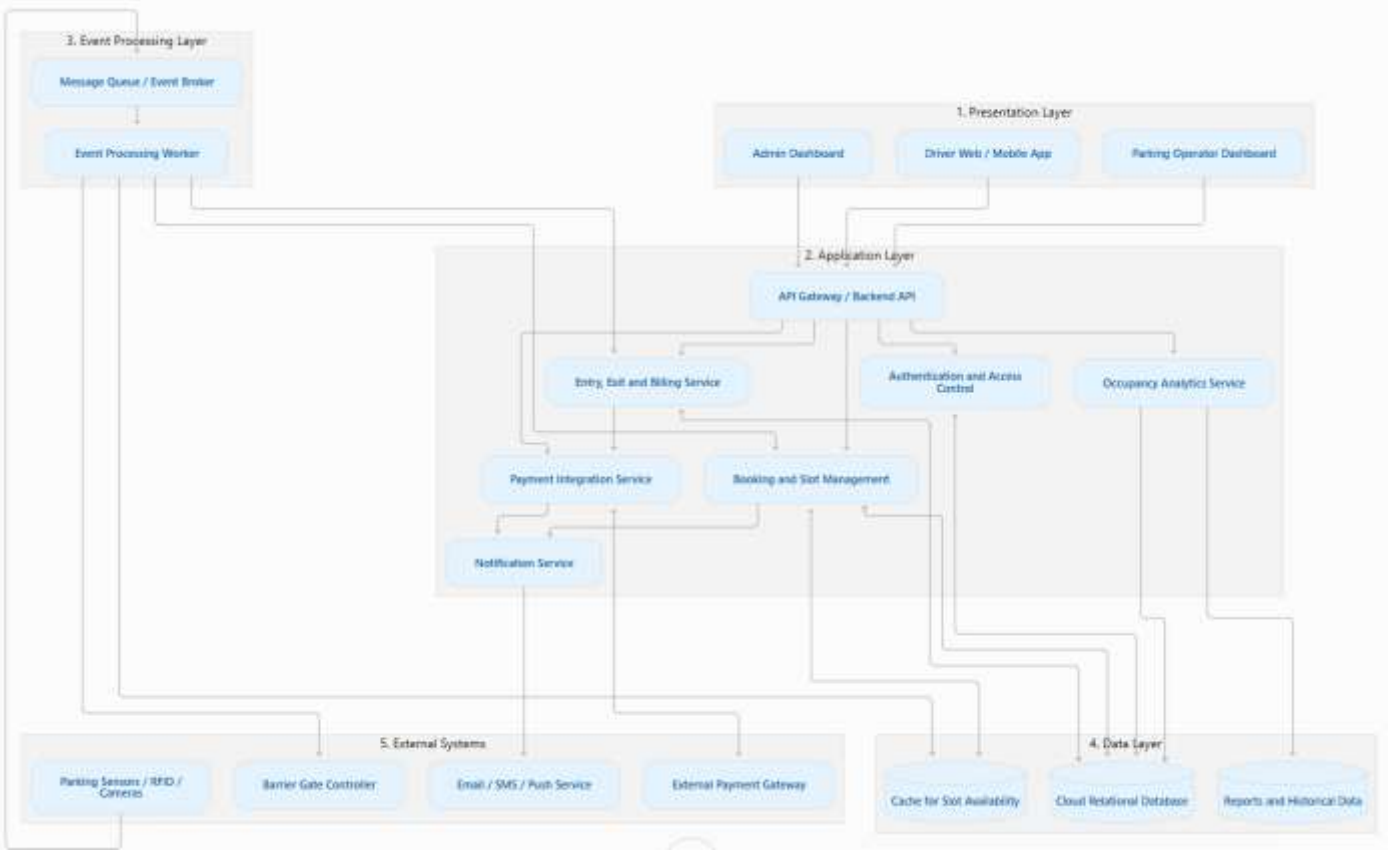


Diagram note: The message queue supports asynchronous events. The database remains the authoritative source for bookings and transactions; the cache is used to speed up availability queries. Sensor events must be validated before changing confirmed slot or booking records.

4. Explain Components and Their Interactions

4.1 Presentation Layer

Components: Driver application, operator dashboard, and administrator dashboard.
Responsibilities:

- Display parking availability.
- Allow drivers to book or cancel slots.
- Display payment details and booking confirmations.
- Provide parking management and analytics screens.

4.2 API Gateway / Backend API

Responsibilities:

- Receive requests from web and mobile clients.
- Route requests to the appropriate backend service.
- Validate incoming requests and apply access controls.
- Return responses to the client.

4.3 Authentication and Access Control

Responsibilities:

- Verify user login credentials.
- Manage user roles such as driver, operator, and administrator.
- Prevent unauthorized access to sensitive functions.
- Apply role-based permissions.

4.4 Booking and Slot Management Service

Responsibilities:

- Search available parking slots.
- Reserve and cancel slots.
- Prevent overlapping bookings.
- Update booking status and slot availability.

4.5 Entry, Exit and Billing Service

Responsibilities:

- Record vehicle entry and exit times.
- Calculate parking duration.
- Apply configured parking rates.
- Generate billing records and release slots after a valid exit.

4.6 Payment Integration Service

Responsibilities:

- Connect to the external payment gateway.
- Initiate digital payment transactions.
- Verify payment confirmations.
- Update payment status and support reconciliation of failed or delayed transactions.

4.7 Event Processing Layer

Components: Message queue and event-processing worker.
Responsibilities:

- Receive sensor and vehicle-entry/exit events.
- Process events asynchronously.
- Send validated updates to relevant services.
- Reduce direct dependency between sensors and application components.

4.8 Database and Cache

Cloud relational database: Stores users, parking slots, bookings, parking sessions, payments, and transaction records.
Cache: Stores frequently requested slot availability to improve response time. It must be refreshed or invalidated when relevant events occur.

Historical data storage: Stores aggregated parking usage information and reports for analytics.

4.9 Analytics and Notification Services

Analytics service:

- Calculates occupancy rates.
- Generates usage and revenue reports.
- Identifies peak parking hours.
- Provides information for better parking-space planning.

Notification service:

- Sends booking confirmations.
- Provides payment status updates.
- Notifies users about cancellations and relevant booking changes.

4.10 External Systems

- Parking sensors: Detect whether a slot is occupied.
- RFID readers or cameras: Help identify authorized vehicles.
- Barrier gate controller: Controls vehicle entry and exit when supported.
- Payment gateway: Processes digital transactions.
- Notification provider: Sends email, SMS, or push notifications.

4.11 Overall System Workflow





The event-processing layer can handle sensor updates and notifications in parallel with the main workflow. Payment and slot-release operations must be confirmed and recorded consistently.

5. Discuss Quality Attributes

Quality Attribute	Architectural Support
Scalability	Application services and event workers can be scaled independently as the number of users and parking locations increases.
Security	HTTPS, authentication, role-based authorization, input validation, and secure payment integration protect the system.
Performance	Caching, database indexing, efficient APIs, and asynchronous event processing reduce response times.
Reliability	Database transactions, unique booking identifiers, duplicate-event handling, and payment reconciliation improve correctness.
Maintainability	Separate layers and modular services simplify testing, debugging, and software updates.
Availability	Cloud hosting, health monitoring, backups, redundancy, and recovery procedures reduce downtime.

5.1 Scalability

The system can support additional parking locations by extending its application and data-processing capacity. Frequently requested availability information can be cached, while event workers can be scaled as sensor activity increases.

5.2 Security

All client-server communication should use HTTPS. Role-based access control limits administrative operations, and passwords must be securely hashed. Payment card information should be handled by a compliant payment provider rather than stored directly in the application database.

5.3 Performance

Caching, database indexing, optimized queries, and asynchronous notifications help provide faster responses. Booking confirmation must still use an authoritative database transaction to prevent conflicting reservations.

5.4 Reliability
Database transactions and validation rules help maintain consistent booking and billing records. Repeated sensor events should be handled idempotently, meaning processing the same event more than once should not create duplicate actions.

5.5 Maintainability
The modular design separates booking, billing, payment, notifications, and analytics. Developers can test and modify individual components without unnecessarily changing the entire application.

5.6 Availability
Cloud deployment, health checks, monitoring, backups, and redundant infrastructure can reduce downtime. If a sensor or external service fails, the system should detect the issue and provide a controlled fallback procedure.

6. Justify Architectural Decisions

6.1 Rationale for Architectural Choices
1. Layered design: Provides clear separation between presentation, business logic, and data management.
2. Client–Server communication: Enables centralized parking information to be accessed from different devices.
3. Event-driven processing: Supports sensor updates and notifications without blocking the main user workflow.
4. Cloud database: Provides centralized storage and supports multiple parking locations.
5. API-based integration: Enables communication with payment gateways, sensors, and other external systems.
6. Caching: Improves the speed of frequent slot-availability queries.

6.2 Architectural Trade-Offs

Advantage	Trade-Off
Modular components improve maintainability.	More components require additional configuration and monitoring.
Event-driven processing supports real-time updates.	Events may be delayed or duplicated, requiring validation and retry handling.
Cloud deployment supports scaling.	Cloud infrastructure introduces recurring costs and dependency on network connectivity.
Caching improves response time.	Cached availability may become temporarily outdated.
External payment integration simplifies payment handling.	Payment-gateway failures can delay transaction confirmation.
Independent services can scale separately.	Distributed communication and troubleshooting become more complex.

6.3 Future Enhancements
1. AI-Based Parking Prediction: Predict parking demand during peak hours.
2. Smart Navigation: Guide drivers to the nearest available slot.
3. Electric Vehicle Support: Add EV charging-slot availability and booking.
4. Dynamic Pricing: Adjust parking rates according to demand and operator policies.
5. Multiple Location Integration: Connect malls, colleges, airports, and other parking facilities through a centralized platform.
6. Mobile Notifications: Provide reminders, booking alerts, and parking-expiry notifications.

6.4 Long-Term Maintainability and Integration
The system uses defined APIs and modular components so that new payment providers, sensor types, parking locations, and analytics features can be added with limited changes to existing modules. Standardized event formats, documentation, automated testing, and monitoring will support long-term maintenance.
For a student project or initial deployment, these components can be implemented as modules in a single backend application. If the system grows significantly, selected components such as notifications, analytics, and event processing can be separated into independently deployed services.

7. Architecture Design Validation

The following checklist validates whether the proposed architecture addresses the assignment requirements.

Validation Criterion	Expected Result
Functional requirements mapped to components	Booking, availability, billing, payments, and analytics are represented.
Quality attributes addressed	Scalability, security, performance, reliability, maintainability, and availability are discussed.
Architecture diagram completeness	Presentation, application, event-processing, data, and external-system components are included.
Component communication	APIs, database interactions, and asynchronous event processing are shown.
Data consistency	Booking validation, transaction records, and duplicate-event handling are considered.
Security and failure handling	Authentication, access control, payment verification, and fallback procedures are included.
Future extensibility	New services, parking locations, and external integrations can be supported.

Validation outcome: The architecture provides a suitable design baseline for the specified parking-management requirements. Before implementation, the project team should confirm sensor-update frequency, expected concurrent users, response-time targets, deployment costs, recovery objectives, and the policies governing booking and billing.

8. Conclusion

The Smart Parking Management System uses a hybrid software architecture to combine layered application design, client-server communication, and event-driven processing. It supports real-time parking availability, advance booking, automated entry and exit billing, digital payments, and occupancy analytics.

The architecture improves modularity, scalability, performance, security, and maintainability while supporting integration with sensors and external payment services. Its modular design also provides a foundation for future enhancements such as AI-based demand prediction, EV charging management, and multi-location parking services. Therefore, the proposed architecture is suitable for developing a reliable and extensible Smart Parking Management System for malls and campuses.